

Lab 13: CDC → Kafka → Downstream Consumer with a Resolved-Timestamp Frontier (75 min)

Learning Objectives

By the end of this lab you will be able to:

- Run a Core changefeed and an Enterprise changefeed and explain when each is appropriate
- Emit to Kafka with `updated`, `resolved`, `diff`, and `key_in_value` options and read the envelopes
- Track the **resolved-timestamp frontier** in a consumer and know when data is safe to act on
- Build an **idempotent consumer** that survives at-least-once delivery and replays
- Measure changefeed lag with `changefeed_max_behind_nanos` and know what makes it grow
- Handle schema changes under an active changefeed without breaking downstream consumers
- Pause, resume, and restart a changefeed from a cursor after a downstream outage

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- Docker (for Kafka). A webhook-sink fallback with `nc` is provided if Docker is unavailable.
- `python3` for the consumer

Setup

1. Kafka

```
mkdir -p /tmp/lab13 && cd /tmp/lab13

cat > docker-compose.yml <<'YML'
services:
  kafka:
    image: apache/kafka:3.9.0
    container_name: kafka
    hostname: kafka
    # Kafka joins the lab cluster's network, because the *database* is what
    # connects to it. A changefeed sink is dialled by the CockroachDB node, not
    # by your shell.
    networks: [crdb]
    environment:
      KAFKA_NODE_ID: "0"
      KAFKA_PROCESS_ROLES: "controller,broker"
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"
      KAFKA_CONTROLLER_QUORUM_VOTERS: "0@kafka:9093"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: "CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

networks:
  crdb:
    external: true
    name: crdb-labs_default
YML

docker compose up -d
docker compose logs -f kafka | grep -m1 "Kafka Server started"
```

The official `apache/kafka` image, not `bitnami/kafka`. Bitnami withdrew its versioned Docker Hub tags, so `bitnami/kafka:3.7` now fails to pull. The Apache image drops the `_CFG` infix from every environment variable and keeps its CLI tools in `/opt/kafka/bin` rather than on `PATH` — which is why the commands below spell out the full path.

⚠ **Start the lab cluster first** (`scripts/crdb up`) — this compose file joins the network that stack creates, and will refuse to start if it does not exist yet.

Why `kafka:9092` and not `localhost:9092` ? The changefeed is opened *by the database node*, inside its own container. `localhost` there is the node itself, not your machine, so a sink of `kafka://localhost:9092` fails with a connection refused that looks baffling until you remember whose `localhost` it is. Advertising `kafka:9092` on the shared network is what makes the sink reachable from where it is actually dialled. Every `kafka-console-consumer` command below runs *inside* the Kafka container (`docker compose exec kafka ...`), which is why those still say `localhost:9092` .

2. Cluster

```
scripts/crdb up
export C='postgresql://root@localhost:26257?sslmode=disable'
```

The cluster runs in Docker (see [Lab 1](#)). From your machine it is `localhost:26257` ; from inside another container it is `crdb1:26257` . `scripts/crdb run ...` executes inside node 1.

Enterprise changefeeds need a licence — and it is free for training. Cockroach Labs issues licences at no cost for training and evaluation, so ask your instructor whether the class has one. Export it before starting the cluster and the wrapper applies it: It ships with the repo (`.license.env`): `scripts/crdb up` prints `enterprise licence applied` when it starts, and nothing further is needed. Without one, `CREATE CHANGEFEED ... INTO` is refused and you run Part A only — core changefeeds, which need no licence and teach the same envelope format.

```
scripts/crdb sql <<'SQL'
SET CLUSTER SETTING kv.rangefeed.enabled = true;

CREATE DATABASE shop;
USE shop;

CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer    STRING NOT NULL,
  total       DECIMAL(12,2) NOT NULL,
  status      STRING NOT NULL DEFAULT 'new',
  updated     TIMESTAMPTZ NOT NULL DEFAULT now()
);
SQL
```

Tasks

Part A: Core Changefeed — the Zero-Infrastructure Version (8 min)

A Core changefeed streams straight to the SQL session. No sink, no license, no delivery guarantees beyond "this session is connected".

1. Terminal A:

```
scripts/crdb sql -e "EXPERIMENTAL CHANGEFEED FOR shop.orders WITH updated;"
```

2. Terminal B:

```
scripts/crdb sql -e "  
  INSERT INTO shop.orders (customer, total) VALUES ('alice', 42.00);  
  UPDATE shop.orders SET status = 'paid' WHERE customer = 'alice';  
  DELETE FROM shop.orders WHERE customer = 'alice';"
```

3. Read the envelopes in terminal A. Note that a delete emits `after: null` — a tombstone.

If you redirect a core changefeed to a file or a pipe, you get nothing. The default `--format=table` buffers all output to compute column widths, so a stream that never ends never prints. Use a streaming format when the output is not a terminal:

```
stdbuf -oL scripts/crdb sql --format=csv \  
-e "EXPERIMENTAL CHANGEFEED FOR shop.orders WITH updated;" > /tmp/cdc.csv
```

Both parts matter. `--format=csv` avoids the column-sizing buffer, and `stdbuf -oL` line-buffers stdout — without it, nothing reaches the file until the process exits, and a changefeed never exits. (On macOS, `stdbuf` comes from `brew install coreutils` as `gstdbuf`; in a terminal you don't need either.) The `key` and `value` columns come back **hex-encoded** (`\x7b2261...`) because they are `BYTES`. Decode them with:

```
python3 -c "import csv,sys  
for r in csv.reader(open('/tmp/cdc.csv')):  
    for c in r:  
        if c.startswith(r'\x'): print(bytes.fromhex(c[2:]).decode('utf-8','replace'))"
```

This is a `cockroach sql` presentation detail, not a changefeed one — an Enterprise changefeed to a real sink emits plain JSON.

Use Core when	Use Enterprise when
Ad-hoc debugging	Anything in production
You need no durability	You need at-least-once delivery
Single consumer, short-lived	Multiple consumers, restartable

Part B: Enterprise Changefeed to Kafka (12 min)

1. Create the changefeed:

```
CREATE CHANGEFEED FOR TABLE shop.orders
  INTO 'kafka://kafka:9092'
  WITH updated, resolved = '10s', diff, key_in_value,
       min_checkpoint_frequency = '10s';
```

Note the returned `job_id`.

2. Confirm the topic exists:

```
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
  localhost:9092 --list
```

The topic is named after the table: `orders`.

3. Generate changes:

```
scripts/crdb sql <<'SQL'
USE shop;
INSERT INTO orders (customer, total) SELECT 'cust-' || g, (g * 3.50)::DECIMAL(12,2)
FROM generate_series(1, 20) g;
UPDATE orders SET status = 'paid', updated = now() WHERE total > 40;
DELETE FROM orders WHERE total < 10;
SQL
```

4. Consume raw:

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic orders --from-beginning
```

Ctrl+C when you have seen enough. A changefeed with `resolved` emits a watermark every few seconds, so the stream never goes quiet — which is why `--timeout-ms` (an *inactivity* timeout) would never fire here.

5. Decode an envelope. With `updated`, `diff`, `key_in_value` you get:

```
{
  "after":
    {"id":"...", "customer":"cust-3", "total":10.50, "status":"paid", "updated":"..."},
  "before":
    {"id":"...", "customer":"cust-3", "total":10.50, "status":"new", "updated":"..."},
  "key":    ["..."],
  "updated": "1716400000000000000.0000000000"
}
```

Option	Adds	Costs
<code>updated</code>	MVCC timestamp of the change	negligible
<code>resolved = 'Ns'</code>	Periodic watermark messages	more messages
<code>diff</code>	The <code>before</code> image	~2× payload size
<code>key_in_value</code>	Primary key inside the value	small
<code>format = avro</code>	Avro + schema registry	needs a registry
<code>envelope = 'wrapped' (default)</code>	before/after wrapper	—

6. Watch the job:

```
SELECT job_id, status, running_status, high_water_timestamp
FROM [SHOW CHANGEFEED JOBS];
```

`high_water_timestamp` is the frontier: every change with an MVCC timestamp at or below it has already been emitted.

Part C: The Resolved-Timestamp Frontier (15 min)

This is the part everyone gets wrong. Changefeed messages arrive **out of order across ranges**. A `resolved` message is the cluster telling you: *no further message will arrive with a timestamp below this value*. Only then is a time window complete.

1. Watch the resolved messages arrive:

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic orders \
  | grep --line-buffered resolved      # Ctrl+C to stop
```

2. Build a frontier-tracking consumer — `/tmp/lab13/consumer.py`:

```
#!/usr/bin/env python3
"""Frontier-tracking, idempotent CDC consumer.

Buffers rows until the resolved timestamp passes them, then applies them
exactly once (keyed by primary key + MVCC timestamp).
"""
import json, sys, subprocess
from collections import defaultdict

frontier = "0"           # highest resolved timestamp seen
pending  = []            # (updated_ts, key, payload) not yet final
applied  = {}            # key -> last applied timestamp (idempotency ledger)

def apply(key, ts, payload):
    # At-least-once delivery means we WILL see duplicates and replays.
    # Applying only when ts > last-applied makes the consumer idempotent.
    if key in applied and ts <= applied[key]:
        print(f" skip duplicate {key} @ {ts}")
        return
    applied[key] = ts
    action = "DELETE" if payload.get("after") is None else "UPSERT"
    print(f" {action:6} {key} @ {ts}")

def handle(line):
    global frontier
    try:
        msg = json.loads(line)
    except json.JSONDecodeError:
        return
    if "resolved" in msg:
        frontier = msg["resolved"]
        ready = [p for p in pending if p[0] <= frontier]
        for ts, key, payload in sorted(ready):
            apply(key, ts, payload)
        pending[:] = [p for p in pending if p[0] > frontier]
        print(f"FRONTIER advanced to {frontier} "
              f"({len(ready)} applied, {len(pending)} still pending)")
        return
    key = json.dumps(msg.get("key"))
    ts = msg.get("updated", "0")
    pending.append((ts, key, msg))

for line in sys.stdin:
    handle(line.strip())
```

3. Run it against the live topic:

```
docker compose exec -T kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic orders --from-beginning \
  | python3 -u /tmp/lab13/consumer.py
```

Leave this running — it prints each `FRONTIER advanced` as it happens. Ctrl+C when done.

4. In another terminal, make changes and watch the frontier advance:

```
scripts/crdb sql -e "
  USE shop;
  INSERT INTO orders (customer, total) VALUES ('frontier-test', 99.99);
  UPDATE orders SET status='shipped' WHERE customer='frontier-test';"
```

The rows sit in `pending` until the next `resolved` message, then apply in timestamp order.

5. The three questions the frontier answers:

Question	Answer
"Is my materialized view up to date as of 14:00?"	Yes, iff frontier ≥ 14:00
"Can I close the books for yesterday?"	Yes, iff frontier ≥ midnight
"Did I miss any events during the outage?"	No, iff you restart from the last frontier

6. Tune the trade-off. Lower `resolved` interval = fresher frontier = more messages:

```
-- Recreate with a 1-second frontier and compare message volume
CREATE CHANGEFEED FOR TABLE shop.orders INTO 'kafka://kafka:9092?topic_prefix=fast_'
  WITH updated, resolved = '1s';
```

Part D: Idempotency and Replay (10 min)

Changefeeds guarantee **at-least-once** delivery, not exactly-once. Duplicates are normal: after a job restart, a lease transfer, or a rebalance, messages are re-emitted.

1. Force a replay — restart the changefeed from an earlier cursor:

```
SELECT job_id, high_water_timestamp FROM [SHOW CHANGEFEED JOBS];
PAUSE JOB <job_id>;
```

```
CREATE CHANGEFEED FOR TABLE shop.orders
  INTO 'kafka://kafka:9092?topic_prefix=replay_'
  WITH updated, resolved = '10s', cursor = '<a timestamp from 5 minutes ago>';
```

2. Run the consumer against the replay topic and confirm your `applied` ledger skips the duplicates it already has.

3. The three ways to be idempotent downstream:

Technique	How	Best for
Upsert by primary key	<code>INSERT ... ON CONFLICT DO UPDATE</code>	Materialized copies
Timestamp guard	Apply only if <code>updated > last_seen</code>	Out-of-order safety
Dedup ledger	Store <code>(key, ts)</code> seen-set with TTL	Side effects (emails, charges)

For side effects, the ledger is the only safe option — you cannot "re-send" an email idempotently by upserting a row.

4. **Verify no data was lost.** Compare the source with what your consumer applied:

```
SELECT count(*) FROM shop.orders;
```

Against the consumer's `applied` map size, minus the deletes.

Part E: Schema Changes Under an Active Changefeed (10 min)

1. **With the changefeed running, add a column:**

```
ALTER TABLE shop.orders ADD COLUMN shipped_at TIMESTAMPTZ;  
INSERT INTO shop.orders (customer, total, shipped_at) VALUES ('post-alter', 12.00, now());
```

2. **Watch the consumer.** The envelope now carries the new field. A consumer that parses strictly (fixed field list) breaks; one that reads by name survives.

3. **Control the behaviour explicitly:**

```
CREATE CHANGEFEED FOR TABLE shop.orders INTO 'kafka://kafka:9092?topic_prefix=strict_'  
WITH updated, resolved='10s', schema_change_policy = 'stop';
```

schema_change_policy	Behaviour
backfill (default)	Emit a full backfill of the table after the change
nobackfill	Continue without re-emitting existing rows
stop	Fail the job so a human decides

4. **The rule for downstream contracts:** additive changes (new nullable column) are safe; renames and type changes are not. Version your consumer's schema handling the same way you'd version an API.

Part F: Lag, Failure & Recovery (10 min)

1. **Measure lag:**

```
-- high_water_timestamp is an HLC DECIMAL; hlc_to_timestamp() converts it.  
-- (A direct ::DECIMAL::TIMESTAMPTZ cast is rejected: 'invalid cast'.)  
SELECT job_id,  
       (now() - hlc_to_timestamp(high_water_timestamp)) AS lag  
FROM [SHOW CHANGEFEED JOBS] WHERE status = 'running';
```

```
curl -s http://localhost:8080/_status/vars | grep changefeed_max_behind_nanos
```

2. **Kill the sink and watch the job react:**

```
docker compose stop kafka
```

```
SELECT job_id, status, running_status FROM [SHOW CHANGEFEED JOBS];
```

The job retries; it does not silently drop events. `protected timestamps` keep the underlying MVCC data from being garbage-collected while the feed is behind.

3. Bring the sink back:

```
docker compose start kafka
```

```
SELECT job_id, status, high_water_timestamp FROM [SHOW CHANGEFEED JOBS];
```

The frontier resumes from where it stopped. Nothing was lost.

The hidden cost: while a changefeed is paused or behind, its protected timestamp prevents GC of old MVCC versions. A changefeed left paused for days grows your storage and slows scans. `SHOW JOBS` is where you find the paused feed nobody remembered.

4. Cancel cleanly:

```
CANCEL JOB <job_id>;  
SELECT job_id, status FROM [SHOW CHANGEFEED JOBS];
```

Cleanup

```
cd lab13 && docker compose down -v && cd ..  
scripts/crdb down
```

Lab 13 Deliverables

□ **Core and Enterprise changefeeds** both run, with a stated rule for when to use each □ **Kafka sink** with `updated`, `resolved`, `diff`, `key_in_value` and the envelopes decoded □ **Frontier consumer** that buffers until resolved and applies in timestamp order □ **Idempotency** proven by replaying from an earlier cursor □ **Schema change** handled under an active feed, with the policy options understood □ **Failure drill**: sink killed and restored with no data loss, lag measured

Challenge Exercises

1. **Avro + schema registry.** Add Confluent Schema Registry to the compose file and re-create the changefeed with `format = avro, confluent_schema_registry = '...'`. What does the consumer gain?
2. **Fan out by tenant.** Use `WITH topic_in_value` and a changefeed on a `REGIONAL BY ROW` table. Can you route each region's events to a different topic?
3. **Exactly-once, end to end.** Combine the dedup ledger with Kafka transactions in your consumer. Where is the remaining window in which a duplicate side effect could occur?
4. **Alert on lag.** Add a Prometheus alert (Lab 9) on `changefeed_max_behind_nanos > 60e9` and prove it fires by pausing the job.

Reference

Command	Purpose
<code>EXPERIMENTAL CHANGEFEED FOR t</code>	Core changefeed to the SQL session
<code>CREATE CHANGEFEED FOR TABLE t INTO 'kafka://...'</code>	Enterprise changefeed
<code>WITH resolved = '10s'</code>	Emit frontier watermarks
<code>WITH diff</code>	Include the <code>before</code> image
<code>WITH cursor = '<ts>'</code>	Start (or replay) from a timestamp
<code>WITH schema_change_policy = 'stop'</code>	Fail on schema change instead of backfilling
<code>SHOW CHANGEFEED JOBS</code>	Status and <code>high_water_timestamp</code>
<code>PAUSE / RESUME / CANCEL JOB</code>	Feed lifecycle
<code>changefeed_max_behind_nanos</code>	Lag metric for alerting